

ATIVIDADE FINAL DA DISCIPLINA DE ALGORITMOS

Programa 01 – Separação de Cadeias de Proteínas (1,0 ponto)

Na pasta **pdbproteins** encontram-se 10 arquivos de proteínas no formato PDB contendo duas ou mais cadeias polipeptídicas.

O objetivo é desenvolver um programa capaz de separar automaticamente cada cadeia em um novo arquivo.

Requisitos

a) Construir um programa que seja capaz de localizar todos os arquivos da pasta **pdbproteins** e imprimir na tela:

- Nome do arquivo
- Caminho completo (path)
- Número total de arquivos encontrados

b) Criar automaticamente uma pasta denominada “chainsproteins” para armazenar os novos arquivos gerados.

c) Abrir cada arquivo PDB e identificar todas as cadeias existentes. Cada cadeia deverá ser salva em um novo arquivo utilizando o formato:

A_nomeOriginal.pdb

B_nomeOriginal.pdb

C_nomeOriginal.pdb

...

onde A, B, C representam os identificadores das cadeias.

d) Ao final da execução, gerar um relatório na tela contendo:

- Nome da proteína
- Número de cadeias encontradas
- Nome dos arquivos gerados

Exemplo:

Proteína: 1ABC.pdb

Cadeias encontradas: 3

Arquivos gerados: A_1ABC.pdb; B_1ABC.pdb; C_1ABC.pdb

Programa 02 – Modularização (1,0 ponto)

Reescrever o Programa 01 utilizando funções/módulos. A funcionalidade de separação das cadeias deverá ser implementada em um módulo independente.

Exemplo: `from separador import separar_cadeias`

O programa principal deverá apenas: localizar os arquivos; chamar a função do módulo; exibir os resultados. Os resultados devem ser idênticos aos do Programa 01.

Programa 03 – Estatísticas Estruturais (1,0 ponto)

Utilizando os arquivos gerados em `chainsproteins`, desenvolver um programa que determine para cada cadeia:

a) Número total de átomos; b) Número total de resíduos; c) Número total de aminoácidos diferentes.

O programa deverá gerar uma tabela semelhante a:

Arquivo	Átomos	Resíduos	Aminoácidos diferentes
A_1ABC	2450	315	19
B_1ABC	1870	240	18

Ao final, identificar: cadeia com maior número de resíduos; cadeia com menor número de resíduos.

Programa 04 – Frequência de Aminoácidos (1,0 ponto)

Para cada cadeia gerada:

1. Contar a frequência de ocorrência dos 20 aminoácidos.
2. Calcular a porcentagem de cada aminoácido.
3. Construir um gráfico de barras.

Exemplo: ALA = 15%; GLY = 8%; LYS = 12%

O gráfico deverá ser salvo automaticamente em uma pasta: `graphs`

Programa 05 – Busca de Motivos Estruturais (1,0 ponto)

Criar um programa capaz de procurar padrões de aminoácidos.

Exemplo:

Se o usuário digitar: GLY-LYS-SER o programa deverá informar: a) em quais proteínas o padrão aparece; b) em quais cadeias; c) posição da ocorrência.

Exemplo:

Proteína: 1ABC

Cadeia: B

Posição: 145

Programa 06 – Análise automática de pockets em proteínas utilizando DoGSite3 (3,0 pontos)

Na pasta `pdbproteins`, encontram-se arquivos de proteínas no formato PDB. O objetivo desta atividade é desenvolver um programa em Python capaz de executar automaticamente o software **DoGSite3** para identificar possíveis pockets (cavidades de ligação) em cada proteína.

Etapa 1 – Instalação do DoGSite3

O DoGSite3 é um programa desenvolvido pela Universidade de Hamburgo para identificação e caracterização de pockets em proteínas.

Download

Acesse o site oficial:

<https://www.zbh.uni-hamburg.de/en/forschung/amd/software/dogsite3.html>

ou

<https://software.zbh.uni-hamburg.de>

Crie uma conta acadêmica, realize o registro solicitado pelos desenvolvedores e faça o download da versão compatível com seu sistema operacional.

Instalação

Siga as instruções fornecidas pelos desenvolvedores para instalação do programa.

Teste

Após a instalação, verifique se o programa está acessível pelo terminal.

Exemplo:

```
dogsite3 --help
```

ou

```
dogsite3 -h
```

Caso o comando seja executado corretamente, a instalação foi concluída com sucesso.

Etapa 2 – Estrutura de Diretórios

Considere a seguinte organização:

ProjetoFinal/

```
|
|
|— programa03.py
|
|— pdbproteins/
|   |— proteina1.pdb
|   |— proteina2.pdb
|   |— proteina3.pdb
```

```
|  └─ ...  
|  
└─ resultados_dogsite/
```

Etapa 3 – Desenvolvimento do Programa

O programa deverá:

- a) Acessar automaticamente a pasta: `pdbproteins` e listar todos os arquivos `.pdb` encontrados.
- b) Criar automaticamente uma pasta denominada: `resultados_dogsite` caso ela ainda não exista.
- c) Executar o DoGSite3 para cada proteína encontrada. Exemplo genérico:

```
dogsit3 -i pdbproteins/proteina1.pdb -o resultados_dogsite/proteina1
```

O comando real poderá variar dependendo da versão instalada.

- d) Criar automaticamente uma subpasta para cada proteína processada.

Exemplo:

```
resultados_dogsite/
```

```
├─ proteina1/
```

```
├─ proteina2/
```

```
├─ proteina3/
```

```
└─ proteina4/
```

- e) Ao final do processamento, o programa deverá informar: a) número total de proteínas analisadas; b) tempo total de execução; c) nome das proteínas processadas; d) existência ou não de erros durante a execução.

Etapa 4 – Tabela Resumo

O programa deverá procurar automaticamente arquivos de resultados gerados pelo DoGSite3 e construir uma tabela denominada: `resumo_geral_pockets.csv`

Quando disponíveis, devem ser extraídas as seguintes informações: a) nome da proteína; b) nome do pocket; c) volume; d) superfície; e) profundidade; f) hidrofobicidade; g) número de aceptores; h) número de doadores.

Etapa 5 – Análise dos Resultados

Responder às seguintes questões:

1. Qual proteína apresentou o pocket de maior volume?
2. Qual proteína apresentou o pocket mais profundo?

3. Existe relação entre volume e profundidade?
4. Quais proteínas apresentaram pockets mais hidrofóbicos?
5. Qual pocket você escolheria para realizar um docking molecular? Justifique.

Etapas 6 – Construção de Gráficos

Construir pelo menos dois gráficos utilizando Python.

Sugestões:

- Volume do principal pocket por proteína;
- Profundidade do principal pocket por proteína;
- Hidrofobicidade do principal pocket por proteína.

Os gráficos devem conter: a) título; b) legenda; c) nomes dos eixos; d) identificação das proteínas.

Referências DogSiteScorer

Volkamer A., Kuhn D., Rippmann F., Rarey M.

DoGSiteScorer: A web server for automatic binding site prediction, analysis and druggability assessment.

Bioinformatics, 2012.

Site oficial: <https://www.zbh.uni-hamburg.de/en/forschung/amd/software/dogsite3.html>

Relatório Final (2,0 pontos)

Cada grupo deverá entregar um PDF contendo:

Introdução

Descrição do problema de cada programa.

Algoritmo

Código fonte, Fluxograma ou pseudocódigo.

Estrutura do programa

Descrição das funções utilizadas.

Resultados

Tabelas e gráficos produzidos.

Uso de IA

Descrever:

- Qual IA foi utilizada;
- Quais prompts foram empregados;
- Quais modificações foram realizadas manualmente.

Critério de avaliação adicional

Durante a apresentação, cada aluno poderá ser solicitado a explicar trechos específicos do próprio código. Caso não consiga explicar o funcionamento de estruturas básicas (laços, funções, listas, dicionários ou leitura de arquivos), a nota do grupo poderá ser reduzida.

Essa última regra costuma ser a forma mais eficaz de garantir que o aluno realmente compreendeu o algoritmo, mesmo quando utilizou IA para auxiliar no desenvolvimento